

```

#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <stdalign.h>
#include <string.h>

// [1] 제현의 XVecPacket430 규격 (16바이트 정렬)
typedef struct alignas(16) {
    uint32_t fmt;    // 0:U64, 1:FP16x4, 2:FP32x2, 3:INT8x8, 4:FP64x1
    uint32_t lanes;  // n개의 64비트 레인 개수 (동적 VL)
    uint32_t flags;  // 옵션 플래그
    uint32_t reserved; // 패딩
} XVecPacket430Header;

// [2] 고도화된 디코더: 패킷 헤더와 인스트럭션 결합
typedef struct {
    uint64_t raw_inst;
    XVecPacket430Header header;
    bool is_matrix;
} xcore_advanced_uop;

void decode_xcore_v3(uint64_t raw, xcore_advanced_uop* uop, uint32_t lanes, uint32_t
fmt) {
    uop->raw_inst = raw;
    // 인입 즉시 판별 (제현의 간략화 사상)
    uop->is_matrix = ((raw & 0x7F) == 0b1011100);

    // 패킷 헤더 설정 (서버에서 넘어온 동적 규격 반영)
    uop->header.fmt = fmt;
    uop->header.lanes = lanes;
}

// [3] Execute: 64-bit Lane 정밀도별 가속 실행 (SIMD 내부 연산)
void execute_xvec_accelerator(xcore_advanced_uop* uop, uint64_t* payload) {
    printf("[Fetch] Inst: 0x%016lX\n", uop->raw_inst);
    printf("[Decode] Format: %d | Lanes: %d\n", uop->header.fmt, uop->header.lanes);

    for (uint32_t i = 0; i < uop->header.lanes; i++) {
        uint64_t data = payload[i]; // 64비트 레인 하나 추출

        // 제현아, 여기서 fmt에 따라 내부적으로 쪼개서 연산하는 게 짤 가속이야 ㅋㅋㅋㅋ
        switch (uop->header.fmt) {
            case 3: // X_FMT_INT8x8: 64비트 하나를 8비트 8개로 쪼개기
                printf(" [Lane %d] INT8x8 가속 모드 가동 중...\n", i);
                break;
            case 1: // X_FMT_FP16x4: AI 연산용 반정밀도 4개 쪼개기
                printf(" [Lane %d] FP16x4 AI 엔진 가동 중...\n", i);
                break;
        }
    }
}

```

```

        default:
            printf(" [Lane %d] Pure 64-bit Native 연산 중...\n", i);
            break;
    }
}
printf("[Commit] %d Lanes 처리 완료 (서버 대응 끝) ㅋㅋㅋㅋ\n\n", uop->header.lanes);
}

int main() {
    // 1. 패킷 데이터 준비 (헤더 + 64비트 페이로드 4개)
    uint64_t payload[4] = {0x0102030405060708, 0x1122334455667788, 0, 0};

    xcore_advanced_uop uop;
    // 2. 명령어 디코딩 (INT8x8 모드, 4레인 설정)
    decode_xcore_v3(0x5B, &uop, 4, 3);

    // 3. 실행 (흥내내기 for문이 아니라 레인 제어 로직)
    execute_xvec_accelerator(&uop, payload);

    return 0;
}

```